

CSE 4113 - Internet Programming Lab

Project Report

Project Name

Team Name

Submitted By

Team Member	Student ID
Member 1	
Member 2	
Member 3	
Member 4	

28th Batch

Department of Computer Science & Engineering
University of Dhaka

Submitted On

[Date]

Project Links

Every link below must be public and live at submission time. The report is evaluated against what these links actually show, not just the write-up.

Item	URL
Public Git Repository	
Deployed Application URL	
API Docs (Swagger/Postman) - public link	
Demo Video	

Table of Contents

Project Links.....	2
Table of Contents.....	3
1. Introduction.....	5
1.1 Problem Definition & Context.....	5
1.2 Target Users & Use Cases.....	5
1.3 Core Features.....	5
1.4 Tech Stack Overview.....	5
2. System Architecture.....	5
2.1 High-Level Architecture Diagram.....	5
2.2 Frontend Architecture.....	5
2.3 Backend Architecture.....	5
2.4 Database Design.....	5
3. API Documentation.....	5
3.1 API Design Overview.....	5
3.2 Endpoint Reference.....	5
3.3 Swagger / Postman Collection Link.....	5
3.4 Sample Request & Response.....	5
4. Authentication & Security.....	5
4.1 Authentication Strategy.....	6
4.2 Authorization & Role Management.....	6
4.3 Security Measures.....	6
4.4 Known Vulnerabilities & Mitigations.....	6
5. Testing & Quality Assurance.....	6
5.1 Testing Strategy.....	6
5.2 Test Coverage Summary.....	6
5.3 Bug Tracking & Resolution Log.....	6
5.4 Sample Test Cases.....	6
6. CI/CD & Deployment.....	6
6.1 Pipeline Overview.....	6
6.2 Environments.....	6
6.3 Deployment Steps.....	6
6.4 Hosting Platform & Live URL.....	6
6.5 Monitoring & Logging.....	6
7. Repository & Documentation Quality.....	6
7.1 Branching Strategy & Commit Conventions.....	6
7.2 README Completeness Checklist.....	6
7.3 Code Organization / Folder Structure.....	6
7.4 Local Setup Instructions.....	6
8. Evaluation & Reflection.....	7
8.1 Web Performance & Core Web Vitals.....	7
8.2 Challenges & Solutions.....	7
8.3 Limitations & Future Work.....	7
8.4 Lessons Learned.....	7

8.5 Individual Responsibility.....7

Appendix - Screenshots / UI Walkthrough.....8

1. Introduction

1.1 Problem Definition & Context

1.2 Target Users & Use Cases

1.3 Core Features

1.4 Tech Stack Overview

List frontend, backend, database, and hosting choices, with a one-line reason for each. Also, clarify State Management Solution (e.g., Redux Toolkit, Context API, Zustand) and Third-Party Services/Cloud Infrastructure (e.g., Firebase Auth/Firestore, Supabase, AWS S3).

2. System Architecture

2.1 High-Level Architecture Diagram

Add all relevant diagrams as you see fit, including sequence or flow diagram.

2.2 Frontend Architecture

Component structure, routing, state management.

2.3 Backend Architecture

Service/controller structure, design patterns used, and why.

2.4 Database Design

ERD and schema. Note normalization decisions and indexes. Include a brief paragraph or sequence diagram showing Data Sync Mechanics - specifically how updates flow from the database server to the UI in real time without manual page refreshes.

3. API Documentation

3.1 API Design Overview

REST or GraphQL, versioning scheme, base URL.

3.2 Endpoint Reference

Method	Route	Description	Auth Required

3.3 Swagger / Postman Collection Link

3.4 Sample Request & Response

4. Authentication & Security

4.1 Authentication Strategy

Session, JWT, or OAuth - and why.

4.2 Authorization & Role Management

4.3 Security Measures

Input validation, sanitization, rate limiting, secrets management. State how you protected the application credentials during development and deployment (e.g., using GitHub Secrets, Vercel/Render environment variables) and verify that no credentials were hardcoded into the codebase.

4.4 Known Vulnerabilities & Mitigations

5. Testing & Quality Assurance

5.1 Testing Strategy

Unit, integration, and end-to-end coverage; tools used.

5.2 Test Coverage Summary

5.3 Bug Tracking & Resolution Log

5.4 Sample Test Cases

6. CI/CD & Deployment

6.1 Pipeline Overview

6.2 Environments

Development, staging, production - how each is configured.

6.3 Deployment Steps

6.4 Hosting Platform & Live URL

Add credentials required to access full functionality.

6.5 Monitoring & Logging

7. Repository & Documentation Quality

7.1 Branching Strategy & Commit Conventions

7.2 README Completeness Checklist

Setup steps, environment variables, run/test commands, screenshots.

7.3 Code Organization / Folder Structure

7.4 Local Setup Instructions

8. Evaluation & Reflection

8.1 Web Performance & Core Web Vitals

provide a basic analysis using built-in browser tools (like Chrome Lighthouse or PageSpeed Insights). Document metrics such as: First Contentful Paint (FCP) / Largest Contentful Paint (LCP). Total Blocking Time (TBT). Strategies used for optimization (e.g., lazy loading images, code-splitting components, asset compression).

8.2 Challenges & Solutions

8.3 Limitations & Future Work

8.4 Lessons Learned

8.5 Individual Responsibility

Member Name	Core Responsibilities & Feature Ownership	Key PRs / Commit Contributions	Estimated % Contribution

Appendix - Screenshots / UI Walkthrough
